

Big Data processing in Cloud

Agenda

Introduction to Big Data in the Cloud

Understanding the fundamentals and challenges

Building Data Processing Pipelines

Learning pipeline stages and methods

Advantages of BigQuery

Understanding what sets BigQuery apart

Google Data Studio for Visualization

1

2

3

4

5

6

7

8

Foundations of Data in Big Data Systems

Exploring data types and storage approaches

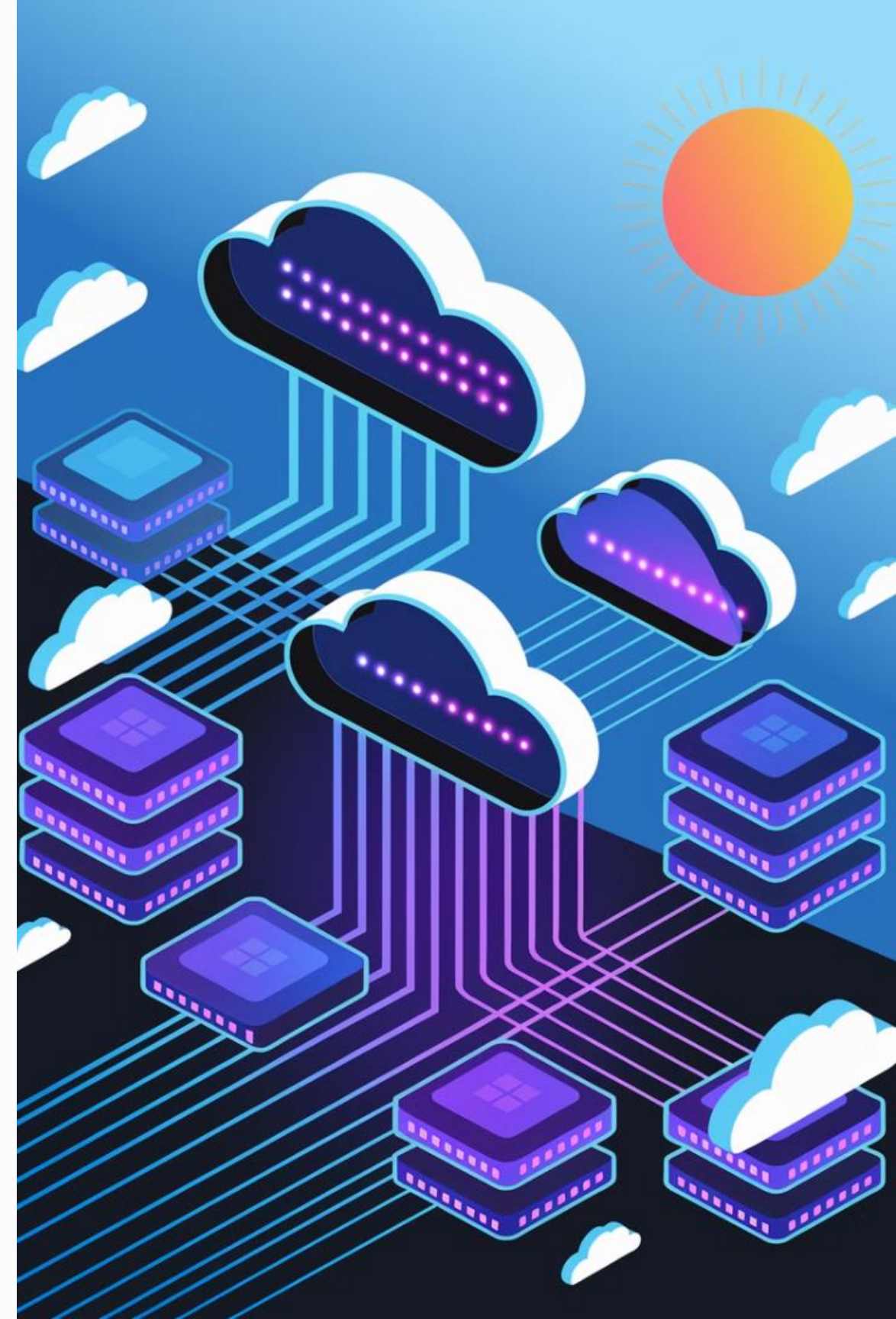
Comparing Data Analytics Across Cloud Providers

Analyzing major cloud solutions

In-Depth Exploration of BigQuery

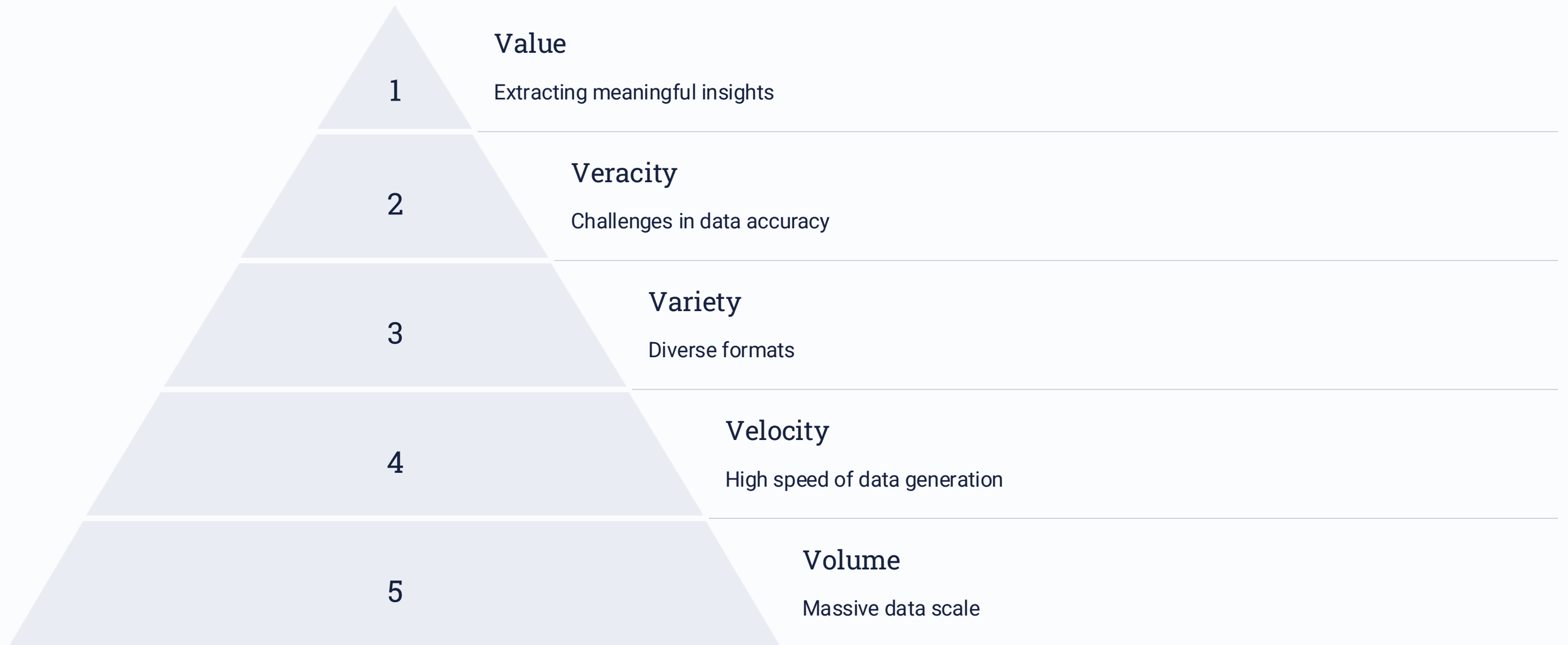
Deep diving into architecture and features

Real-World Applications



Introduction to Big Data in the Cloud

Understanding Big Data



Big Data refers to datasets so large and complex that traditional data processing tools are inadequate. Examples include Twitter's billions of tweets, smart city sensor data, and e-commerce logs.

Challenges of Traditional Infrastructure

Traditional infrastructure creates several obstacles for big data processing:

Scaling Limitations

Limited by physical hardware capacity, making expansion slow and costly compared to cloud solutions.

Maintenance Burden

Ongoing issues with hardware failures, software updates, and security patches require constant attention.

Financial Constraints

High CAPEX for purchasing servers, storage, and networking equipment creates significant upfront costs.

Delayed Time to Market

Long delays due to procurement and deployment cycles (months vs. days on Cloud).

Expertise Requirements

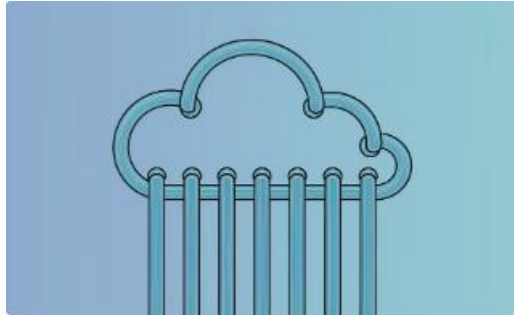
Requires skilled IT staff for setup and management of complex infrastructure.

Inefficient Resource Utilization

Over-provisioning wastes resources; under-provisioning causes performance bottlenecks.

Impact: These challenges ultimately hinder organizational agility and scalability for growing data needs.

Benefits of Cloud for Big Data



Elasticity

Dynamically scale resources (e.g., add CPUs during peak traffic, scale down afterward).



Pay-as-you-go

Pay only for what you use, converting CAPEX to OPEX (e.g., no upfront hardware costs).



Managed Services

Providers handle maintenance, updates, and security (e.g., AWS manages S3).



High Availability

Built-in redundancy and disaster recovery (e.g., data replicated across regions).



Global Reach

Access to worldwide data centers for low latency (e.g., Google's 30+ regions).



Innovation

Instant access to advanced tools like AI/ML without reinvestment (e.g., Google's TensorFlow).

Foundations of Data in Big Data Systems

Exploring Data Types in Big Data

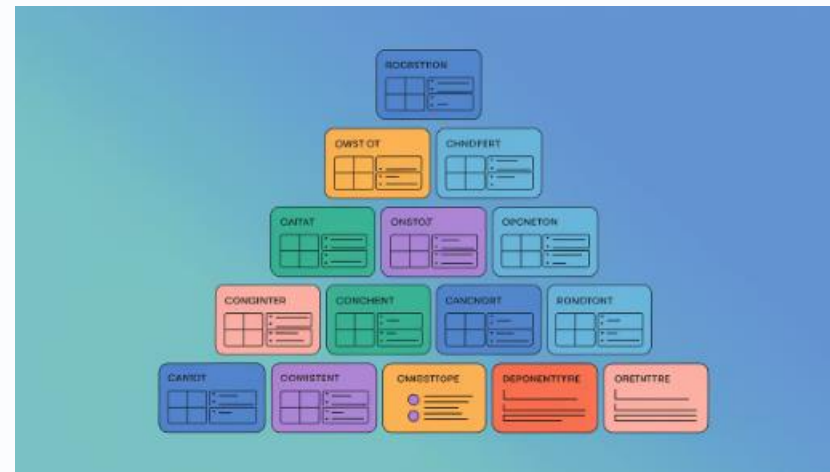
Structured

Organized data in relational databases.
Example: customer records in SQL tables.



Semi-structured

Flexible formats like JSON or XML.
Example: web logs or configuration files.



Unstructured

Unorganized data like text, images, or videos.
Example: YouTube content.



Database and Processing Paradigms

SQL Databases

Relational databases with fixed schemas. Example: MySQL.

- Structured query language
- ACID compliance
- Fixed schema

NoSQL Databases

Non-relational databases for scalability and variety. Example: MongoDB.

- Flexible schema
- Horizontal scaling
- Various data models

Processing Methods

Batch: Process large datasets periodically.

Streaming: Handle data in real-time.

- Batch for historical analysis
- Streaming for immediate insights

Data Lakes vs. Data Warehouses: Key Differences

The modern data architecture pipeline showing progression from raw data to analytics:

Data Lakes

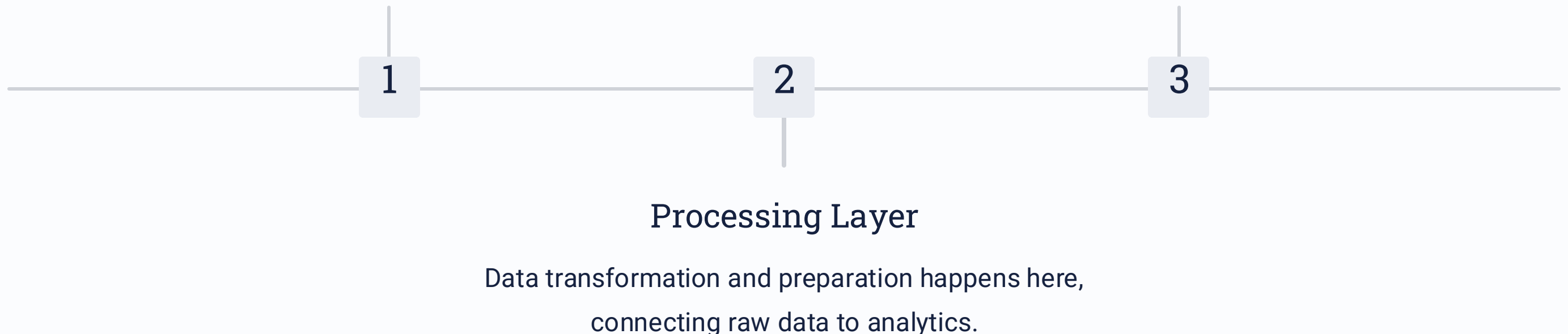
Store raw, unprocessed data of all types. Example: raw IoT sensor data in Hadoop.

- Flexible, cost-effective for storage
- Requires processing for analysis

Data Warehouses

Store processed, structured data for analytics. Example: sales data in BigQuery.

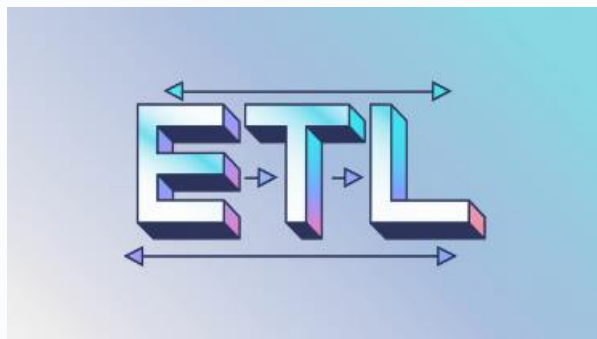
- Optimized for querying, business intelligence
- Less flexible for raw data



Data Processing and Database Concepts

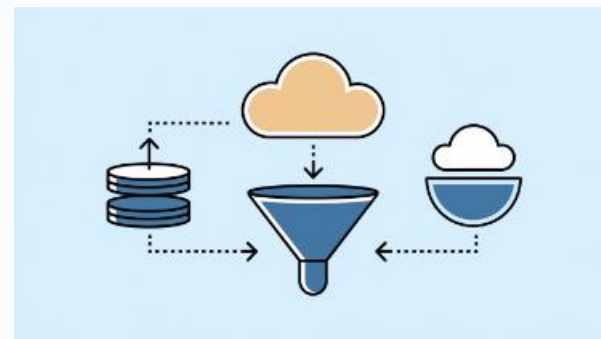
ETL Process

Extract, Transform, Load:
Transform data before loading. Example: cleaning data for a database.



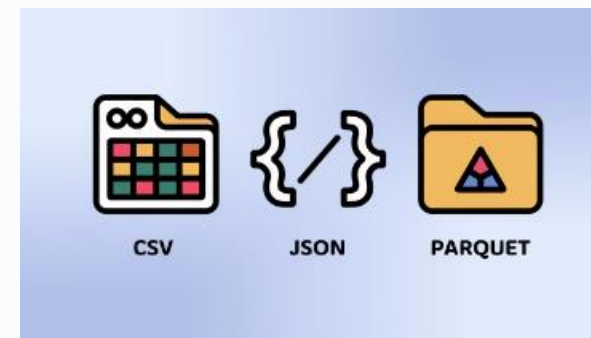
ELT Process

Extract, Load, Transform:
Load raw data first, transform later. Example: BigQuery's approach.



Data Formats

CSV (simple), JSON (flexible), Parquet (optimized); Compression: gzip, Snappy for efficiency.

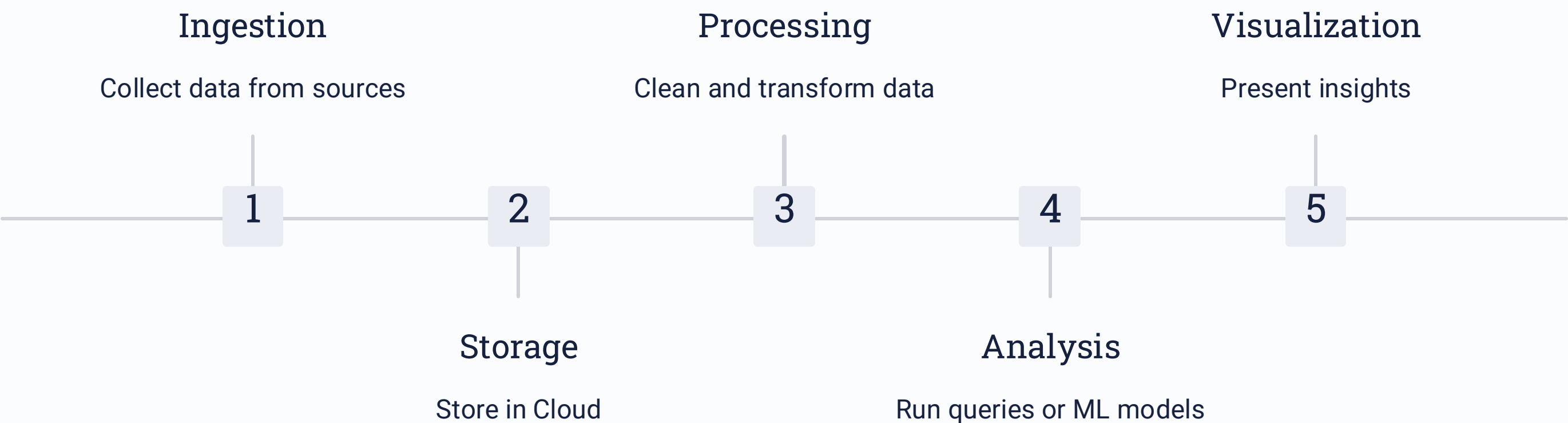


Database Types

OLTP: Handles real-time transactions. **OLAP:** Supports complex analytical queries.



Stages of a Data Processing Pipeline



Data flows through these stages to transform raw information into actionable outcomes. Each stage has specialized tools and techniques.

Processing Approaches and Technologies

Batch vs. Streaming Processing

Batch for historical analysis, Streaming for real-time insights.

Example: Kafka streams live Twitter data for trend analysis.

Core Processing Technologies

- Hadoop: Distributed storage and processing framework.
- Spark: In-memory data processing engine for speed.
- Kafka: Real-time data streaming platform.

Cloud Optimizations

Serverless (e.g., Google Dataflow), Autoscaling for dynamic workloads.

Big Data Across Cloud Providers

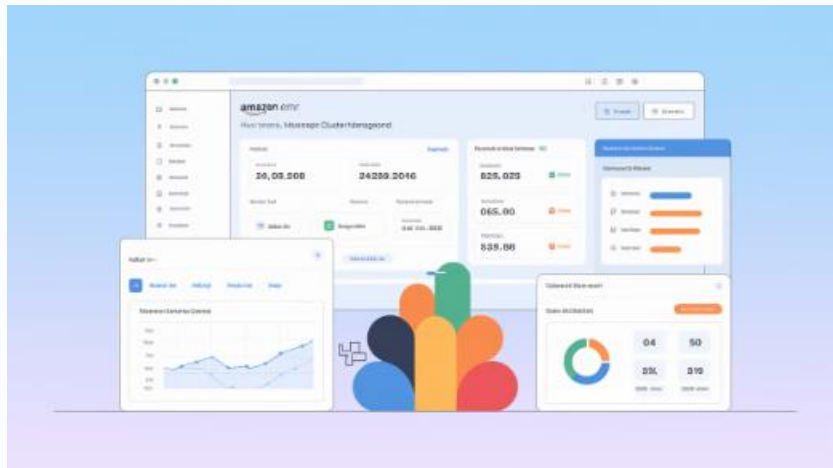
Big Data Services: Processing, Analytics, and Visualization

Category	AWS	Azure	Google Cloud
Data Processing Services	Amazon EMR (Elastic MapReduce) Managed Hadoop, Spark ecosystem - Supports batch, interactive & streaming - Integrates with AWS data services (e.g., S3, Redshift)	Azure HDInsight Managed Hadoop, Spark service- Supports various open-source frameworks (e.g., Kafka, Hive) Integrates with Azure storage (e.g., Blob Storage)	Google Cloud Dataproc Managed Spark and Hadoop service Fast cluster creation (< 90 seconds) Native integration with GCP services (e.g., BigQuery)
Data Analytics/Warehouse	Amazon Redshift Petabyte-scale columnar storage MPP (Massively Parallel Processing) Redshift Spectrum for S3 queries	Azure Synapse Analytics Unified analytics platform Integrates SQL and Spark engines Serverless and dedicated options	Google BigQuery Serverless data warehouse Separation of storage and compute Built-in ML capabilities
Data Visualization	Amazon QuickSight Cloud-native BI service - ML insights with QuickSight Q - Pay-per-session pricing	Power BI Comprehensive analytics solution Deep Microsoft ecosystem integration Cloud and desktop versions	Google Looker & Data Studio Looker: Enterprise BI Data Studio: Shareable dashboards Real-time collaborative editing
Data Storage	- Amazon S3 (object)/ Amazon EBS (block) - Amazon EFS (file) Amazon FSx (managed file systems)	- Azure Blob Storage (object)/Azure Files (file)/ Azure Data Lake Storage Gen2/ Azure Disk Storage (block)	- Google Cloud Storage (object) - Persistent Disk (block) - Cloud Filestore (file) - Hyperdisk (performance block)
Data Lake	- S3 (foundation) / AWS Lake Formation/ AWS Glue (data integration)/Amazon Athena (query service)	- Azure Data Lake Storage Gen2/- Azure Data Factory/ Azure Purview (governance) - Azure Databricks	- Cloud Storage (foundation)/ Dataflow - Dataprep/Cloud Composer (orchestration)

Data Analytics/Warehouse

AWS

Amazon Redshift is a scalable, cloud-based data warehouse for high-performance analytics on structured data.



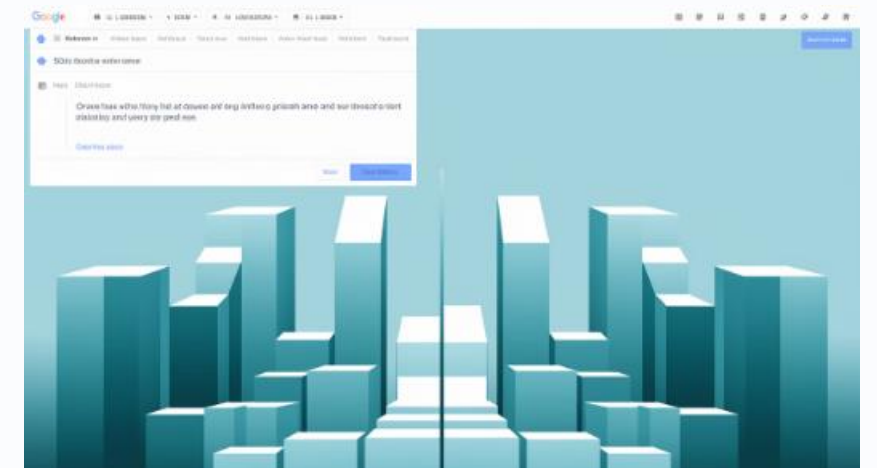
Azure

Synapse Analytics: Unified platform with Spark and SQL for comprehensive analytics.



Google Cloud

BigQuery: Serverless data warehouse designed for large-scale analytics with SQL.



Data Analytics/Warehouse Services and Recommended Use Cases

Criteria	AWS - Amazon Redshift	Azure - Synapse Analytics	Google Cloud - BigQuery
Service Description	Data warehouse for analytics with columnar storage and SQL querying.	Unified analytics platform with Apache Spark and SQL for big data and data warehousing.	Serverless data warehouse for large-scale data analytics with SQL.
Strengths	- High-performance querying for structured data. Seamless integration with AWS ecosystem (S3, EMR). Scalable with cluster management.	- Unified platform for batch and streaming.- Deep integration with Azure Data Lake and Power BI- Built-in AI and machine learning.	- Fully managed and serverless. High-speed querying with columnar storage. Advanced AI/ML integration (e.g., BigQuery ML).
Cost	Pay-as-you-go with cluster costs (e.g., ~\$0.25/hour per node).	Tiered pricing with storage costs (e.g., ~\$1.20/hour + storage).	Per-TB scanned (~\$5/TB).
Performance	Optimized for complex analytical queries with tuning.	Fast with Apache Spark integration.	Optimized for large analytical queries.
Ease of Use	Moderate, requires tuning expertise.	Moderate, benefits from Azure familiarity.	Simplest, ideal for beginners.
Recommended Use Cases	- Best for enterprises needing a robust data warehouse (e.g., retail sales analytics). Scenario: Complex analytical queries on structured data.	- Best for enterprises using Microsoft tools or integrated analytics (e.g., financial institution with Power BI). Scenario: Real-time fraud detection with historical trends.	- Best for startups or analytics-focused teams (e.g., digital marketing firm). Scenario: Analytical queries on structured/semi-structured data (e.g., ad performance).

Why BigQuery Stands Out

1 Serverless Architecture

No need to manage infrastructure. Servers automatically scale to match your workload.

2 Massive Scalability

Handles petabytes of data effortlessly. Queries run across thousands of servers in parallel.

3 Cost-Effective Model

Pay-per-query pricing at approximately \$5 per TB scanned. No idle resource costs.

4 Ideal Use Case

Perfect for businesses needing quick analytics without infrastructure setup or maintenance overhead.



In-Depth Exploration of BigQuery



Introducing BigQuery

Definition

A fully managed, serverless data warehouse for analyzing large datasets using SQL.

Purpose

Enables SQL-based analytics on petabytes of data without infrastructure management.

Example

Analyzing global sales data across years and regions in seconds rather than hours.

Deep Dive into BigQuery Architecture

BigQuery's architecture consists of four key components working together to deliver high-performance analytics.

Dremel

Parallel query execution engine processing queries across thousands of servers.

Colossus

Distributed file system separating compute from data for scalability.

Capacitor

Columnar storage format enabling efficient compression and query processing.

Storage-Compute Separation

Allows independent scaling for optimal cost and performance.

Benefit: Handles complex queries on massive datasets efficiently.

Pros and Cons of BigQuery

Advantages

- High-speed querying using columnar storage
- Pay-per-query pricing (\$5/TB scanned)
- Seamless integrations with Data Studio, TensorFlow, Looker
- Zero infrastructure management

Challenges

- Not optimized for real-time streaming (use Dataflow instead)
- Potential cost overruns if queries are unoptimized
- Less flexible for custom processing frameworks
- SQL knowledge required for effective use

Key Features and Techniques

BigQuery offers several optimization features to improve performance and reduce costs:

Partitioning

Divide tables by date or key for faster queries and reduced costs.

Clustering

Organize data within partitions for efficient filtering and aggregation.

Caching

Reuse query results to reduce costs when running identical queries.

Query Optimization

Avoid SELECT *, use LIMIT, leverage partitioning to minimize data scanned.

Example Query:

```
SELECT country, COUNT(*) as user_count
FROM bigquery-public-data.samples.github_nested
GROUP BY country
LIMIT 10;
```

Partitioning, Clustering, and Best Practices in BigQuery

Partitioning

Divide tables by date, range, or integer to improve query performance and reduce costs.

- Example: Partition a sales table by date (sales_20250301) to query only recent data.

Clustering

Organize data within partitions by frequently filtered columns (e.g., region, product) for efficient filtering.

- Example: Cluster a sales table by region to speed up regional queries.

Best Practices

- Optimize queries with WHERE clauses and LIMIT to minimize data scanned.
- Use partitioned and clustered tables to lower costs.
- Regularly review and archive old data to manage storage.
- Avoid SELECT *; specify only needed columns.

Securing BigQuery

Access Control

Use Identity and Access Management (IAM) to assign roles (e.g., Viewer, Editor).

Data Encryption

Data encrypted at rest (Google-managed keys) and in transit (SSL/TLS).

Row-Level Security

Apply policies to restrict access to specific rows (e.g., by department).

Auditing

Enable audit logs to track access and changes.

Best Practice: Regularly review IAM policies and enable audit logging.

Monitoring and Logging in BigQuery

Monitoring

Use the BigQuery Admin Panel to track query performance, resource usage, and costs.

Logging

Enable Cloud Logging to record query details, errors, and access events.

Alerts

Set up notifications for cost thresholds or performance issues via Cloud Monitoring.

Example & Best Practice

Example: Monitor a query that scans 1TB and costs \$5, setting an alert at \$10.

Best Practice: Review logs weekly to optimize performance and security.

Machine Learning with BigQuery ML

Build and deploy ML models directly within BigQuery using SQL.

Features:

- Linear Regression, Logistic Regression, and K-means clustering.
- Integration with TensorFlow for advanced models.

Example: Predict customer churn with a query:

sql

```
CREATE MODEL project.dataset.churn_model OPTIONS(model_type='logistic_reg') AS SELECT * FROM churn_data  
WHERE label IS NOT NULL;
```

Use Case: Retailers predict demand or marketers target ads.

Best Practice: Start with small datasets to test models before scaling.

Visualizing Big Data with Google Data Studio

Introduction to Google Data Studio

A free, web-based tool for creating interactive, customizable dashboards and reports from Big Data. Visualizes data from BigQuery, Google Analytics, and other sources for actionable insights.



Interactive Dashboards

Drag-and-drop interface for creating various charts including bar, pie, and geo maps.



Real-time Updates

Data updates in real-time with built-in collaboration features for team reporting.



BigQuery Integration

Seamless connection with BigQuery for advanced analytics visualization.
Example: Create a dashboard to track e-commerce sales trends from BigQuery data.

Integrating BigQuery with Data Studio

Steps:

1. Open Data Studio (datastudio.google.com) and create a new report.
2. Add a Data Source – Select BigQuery and authorize access.
3. Choose a BigQuery dataset or table (e.g., bigquery-public-data.samples).
4. Build visualizations (e.g., line chart for sales over time).

Benefits:

- Seamless data flow from BigQuery to interactive reports.
- Supports complex queries and large datasets.
- **Use Case:** Visualize marketing campaign performance metrics in real-time.

Using Data Studio for Big Data Insights

Use Cases:

- Marketing: Track ad performance across channels using BigQuery data.
- Business Intelligence: Monitor sales trends with real-time dashboards.
- Collaboration: Share reports with teams for data-driven decisions.

Best Practices:

- Use filters and date ranges for dynamic reports.
- Apply consistent branding (colors, logos) for professionalism.
- Optimize performance by limiting data refreshes.

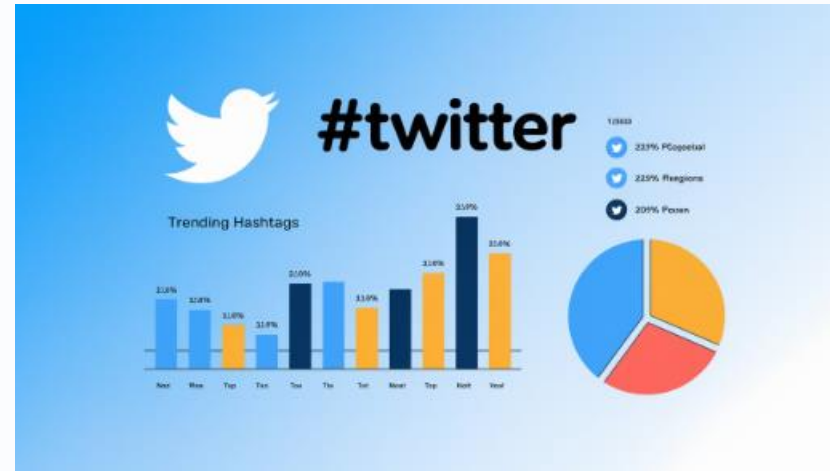
Example: A retailer uses Data Studio to visualize daily sales from BigQuery, adjusting filters for regional insights.

Practical Applications of Big Data



Spotify

Analyzes user listening habits to recommend personalized playlists using BigQuery and ML.



Twitter

Processes billions of tweets to detect trends in real-time using Kafka and BigQuery.



Retail

Forecasts demand using historical sales data through BigQuery analytics.